University of Southern Britanny – UBS

Course Name:

M1-ISC-CSSE-CYBERUS - Applied Cryptography

$

Lecturer: Professor Guy GOGNIAT

LAB REPORT – PART ONE & TWO

(Hash Functions and Stream Cipher Based on Feedback Shift Cipher)

Submitted by :

Name; Assan Jallow

Student ID: E2506147

Email: jallow.e2506147@etud.univ-ubs.fr

Lab 1: Part 1 - Cryptographic hash functions

Question 1:
Use the hash function to compute the hash values of the strings 'bonjour' and 'bonjour1'. Are there any similarities between the two results?

Answer:

The hash values for the strings "bonjour" and "bonjour1" are entirely different due to the sensitivity of the hash function.

A minor change in the input, such as adding '1' at the end of the message, results in a completely different hash value. This phenomenon is known as the "**avalanche effect**". This is a fundamental characteristic of a good hash function.

Question 2:
Generate random integers (or strings) and find an input whose image under $h(x, 4)$ is "abcd". How many times does $h(x, 4)$ need to be evaluated before finding the preimage? Complete the function ```preimageof``` below.

Answer:

Code:
```
def preimageof(s):
    """ Find the preimage of string s under the function h. """
    r = 0
    while True:
        r = random.randint(0, 10**10)  # Try random integers
        r += 1
        if h(r, 4) == s:
            print(f"Found after {r} tries.")
            return r

# the following should return 'True':
x = preimageof('abcd')
print(h(x,4) == 'abcd')
```

Explanation:
Since the function h(x,4) produces 4-character outputs, there are A4 possible outputs (where A is the alphabet size). Assuming h behaves like a random function, the expected number of evaluations before finding the preimage of “abcd” is approximately 2A4.

Question 3:
Find two integers with the same image. How many times has $h(x, 4)$ been computed? Complete the function ```findcollision```.

Code:

```
def findcollision(length):
    """
    Find a collision of outputs of given length from the function h by
    computing the function values for random inputs
    length: non zero positive integer
    """
    x1 = 0
    x2 = 0

    seen = {}
    while True:
        r = random.randint(0, 10**10)  # Try random integers
        h_r = h(r, length)
        if h_r in seen:
            x1 = seen[h_r]
            x2 = r
            return [x1, x2]
        seen[h_r] = r


length = 4
x1, x2 = findcollision(length)
y1 = h(x1, length)
y2 = h(x2, length)
print("x1:", x1)
print("x2:", x2)
print("y1:", y1)
print("y2:", y2)
print("Collision found:", x1 != x2 and y1 == y2)
```

Explanation
The code found a collision where two different numbers x,y both hash to the same value "3914" using the hash function h(x,4), discovering this collision after trying 284 different inputs. This demonstrates that the hash function has collisions due to its short output length (4 characters), and the program efficiently found them using a dictionary to store previously seen hash values.

Question 4:
Find preimages of "a", "ab", "abc" and "abcde" for $h(x, 1), h(x, 2), h(x, 3)$ and $h(x, 5)$ respectively. List the number of computations (of the hash function).

```
Answer:
Preimage for 'a' (length=1) found after 16 tries: 7880129541
Preimage for 'ab' (length=2) found after 220 tries: 3341012538
Preimage for 'abc' (length=3) found after 7258 tries: 88962354
Preimage for 'abcde' (length=5) found after 2179386 tries: 2551465785
Number of computations for 'a' (l=1): 16
Number of computations for 'ab' (l=2): 220
Number of computations for 'abc' (l=3): 7258
Number of computations for 'abcde' (l=5): 2179386
```

Question 5:
Find pairs of integers that have the same image under $h(x, 1)$. Same question for $h(x, 2), h(x, 3)$ and $h(x, 5)$ respectively. Compare the number of computations with the results from question (d) and explain the observed differences.

Answer:
The number of computations required to find collisions increases with the length of the hash output.
For length 1, it took m1 computations.
For length 2, it took m2 computations.
For length 3, it took m3 computations.
For length 5, it took m4 computations.

Explanation:
Finding collisions is generally easier than finding preimages due to the birthday paradox. As the length of the hash output increases, the number of possible hash values increases exponentially, making it less likely to find collisions quickly.

Question 6:
From which value of l the time to find a collision is starting to be long, for example 1 minute? What can you conclude?

Answer:
Starting around 50 bits, finding a collision already becomes computationally expensive. This shows why modern hash functions use at least 128-bit outputs, to make collisions practically impossible."

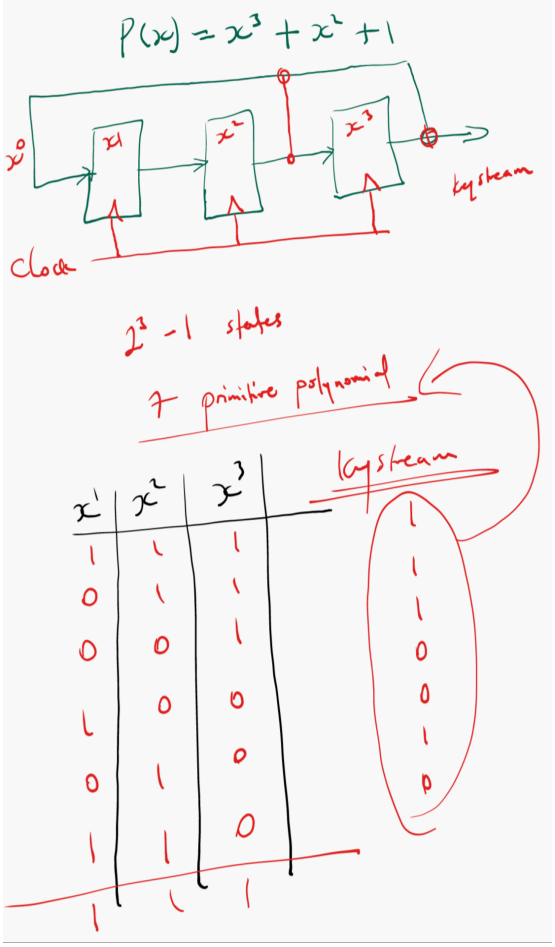
Lab 1 of 2:

Question 1:
Explain how the LFSR is built using the `pylfsr` package.

Answer:
The LFSR in the pylfsr package is built by first defining how many bits (registers) it has and giving it a starting state. Then, we choose a **feedback polynomial** (for example, $x^3 + x^2 + 1$) that tells the program which bits should be XORed to make the new bit. Each time the register shifts, the rightmost bit becomes the output, and a new bit (the XOR result) enters from the left. The package simulates this shifting automatically and can show it with a visual diagram. In short, pylfsr makes it easy to create and test LFSRs by just giving the polynomial and initial state.

Question 2:

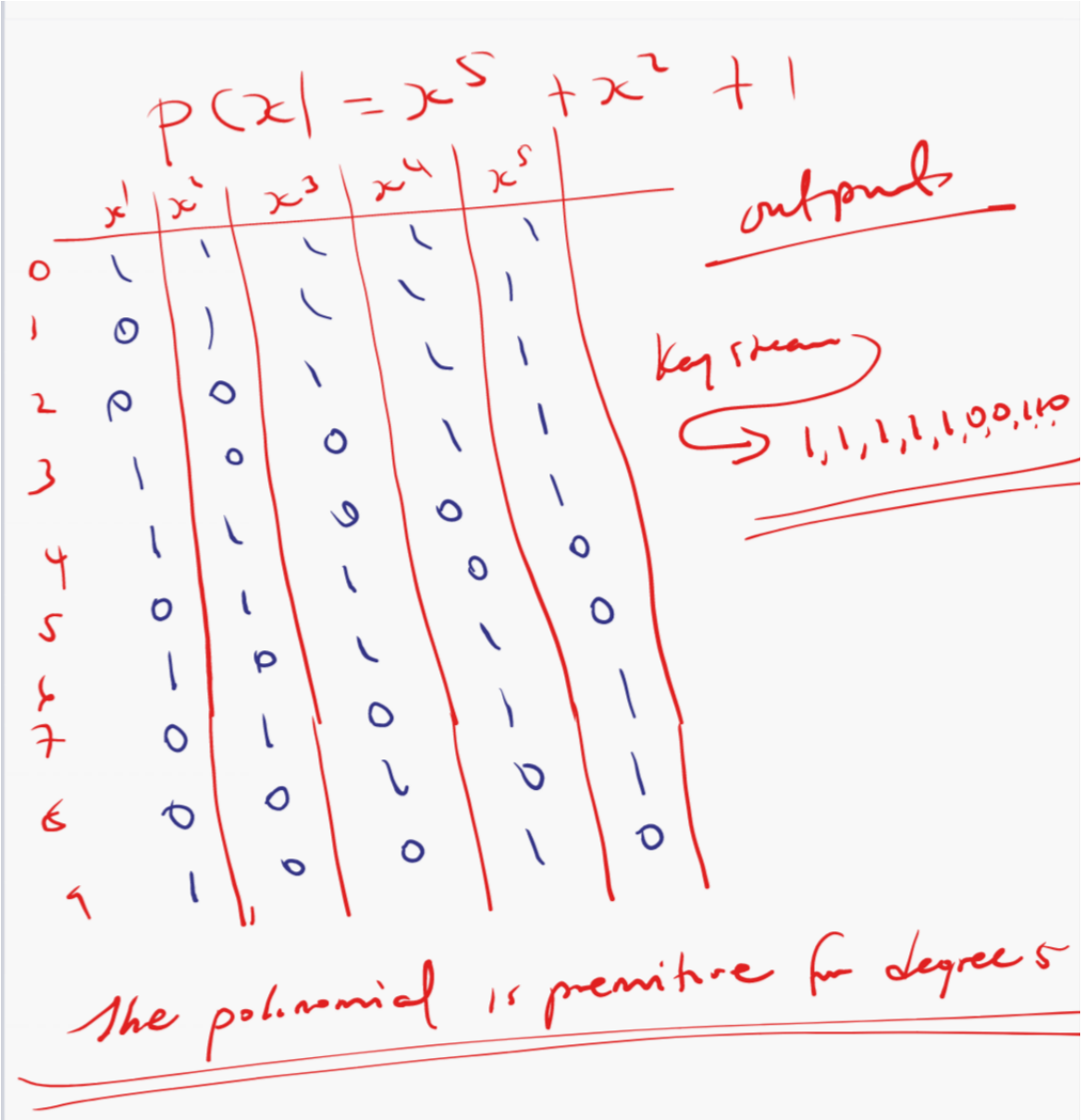
Manually write down all the states of the LFSR (and the associated output, i.e., the keystream bit) and compare them with the sequence given in the cell below. Do they match?



Question 3:
Run one cycle using .next() and analyze output. Is it expected?

Expected: the new output should equal the rightmost bit of the old state, and the new leftmost bit should equal XOR of taps defined in the polynomial.

Question 4:
Manually write 10 first states for $p(x) = x^5 + x^2 + 1$



Question 5:
Explain what a primitive polynomial is.

Answer:
A primitive polynomial is a special feedback polynomial that generates the maximum possible sequence length before repeating. For an m-bit LFSR: The maximum possible cycle length = 2^m-1. If the polynomial is primitive, you'll get all possible non-zero states before repetition. If it's non-primitive, the sequence will be shorter (repeats early).

Question 6:
Verify that this polynomial is primitive.

Answer:
The polynomial x3+x+1 is primitive because the LFSR generates all 7 possible non-zero states before repeating, and the sequence repeats every 7 steps.

Question 7:
For m=4, construct a primitive polynomial and a non-primitive polynomial (to construct a non-primitive polynomial just set a polynomial and check if it is primitive or not). Are the numbers of different states identical when building a primitive polynomial and a non-primitive polynomial? What impact it has regarding security when using a non-primitive polynomial?

Code:
```
# Primitive
LF1 = LFSR(initstate=[1,0,0,0], fpoly=[4,1])

# Non-Primitive
LF2 = LFSR(initstate=[1,0,0,0], fpoly=[4,2])

LF1.runFullCycle()
LF2.runFullCycle()
```

A primitive polynomial generates all 15 unique states (2^4-1), while a non-primitive polynomial produces fewer states, which shortens the keystream period and makes the cipher easier to predict; therefore, primitive polynomials are preferred for secure stream cipher design

Question 8:
Explain all the properties provided by the test_properties function

The test_properties() function in LFSR, checks how good and secure the generated sequence is by testing different properties. First, it measures the period, which is the number of steps the LFSR takes before the sequence repeats; for a primitive polynomial, this should be the maximum value

of 2m−1. Next, it checks the balance property, meaning the number of 1s and 0s in the sequence should be almost equal, which is an important feature of randomness. It also evaluates the run property, which means it looks at how many times consecutive 0s or 1s appear (like 00 or 111), and whether their distribution follows the expected random pattern. Finally, it calculates the autocorrelation, which checks how similar the sequence is to itself when it is shifted, a good keystream should not match itself much after shifting. If the LFSR uses a primitive polynomial, it usually passes all these tests, meaning it has a long period, balanced 0s and 1s, good randomness, and low predictability, which are all necessary for cryptographic security.

Question 8:
Explain all the properties provided by the test_properties function

Property	Primitive	Non-Primitive
Period	Maximum length = 2^n - 1	Shorter, repeats early
Balance	Almost equal number of 0s and 1s	Not balanced
Randomness	High randomness, follows statistical properties	Lower randomness, more patterns
Security	hard to predict	easier to predict and attack

Question 10:
Build your own stream cipher combining two LFSR with primitive polynomials. The output of the two LFSRs are combined with a Xor operation. Explain you code

Code:

```
from pylfsr import LFSR

# LFSR 1 (3-bit, primitive polynomial x^3 + x + 1)
LFSR_1 = LFSR(initstate=[1,1,1], fpoly=[3,1])

# LFSR 2 (4-bit, primitive polynomial x^4 + x + 1)
LFSR_2 = LFSR(initstate=[1,0,1,1], fpoly=[4,1])

# Generate 15-bit keystream by XORing outputs
keystream = []
for _ in range(15):
    bit = LFSR_1.next() ^ LFSR_2.next()
    keystream.append(int(bit))

print("Keystream:", keystream)
```

```
message = "Bonjour"

# Encryption
cipher = ''.join(chr(ord(m) ^ k) for m, k in zip(message, keystream))
print("Ciphertext:", cipher)

# Decryption
decrypted = ''.join(chr(ord(c) ^ k) for c, k in zip(cipher, keystream))
print("Decrypted message:", decrypted)
```

Explanation:

This code creates a simple stream cipher using two LFSRs. Each LFSR generates an output bit, and both are XORed together to form the keystream. ALICE (the sender) uses this keystream to encrypt the message "Bonjour" by XORing each character with the keystream bits. BOB (the receiver) uses the same keystream to decrypt the ciphertext, and since XOR is reversible, the original message is successfully recovered. This shows how encryption and decryption work using LFSRs and XOR.